

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

CACHING TRONG MICROSERVICE

TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

Microservices Distributed Caching

Microservices Distributed Caching Patterns and Practices

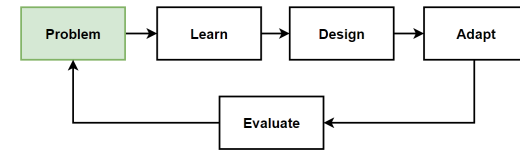
Cache-Aside Pattern

Caching strategies

Cache invalidation

Cache consistency

Problem: Database operations are expensive, low performance



Considerations

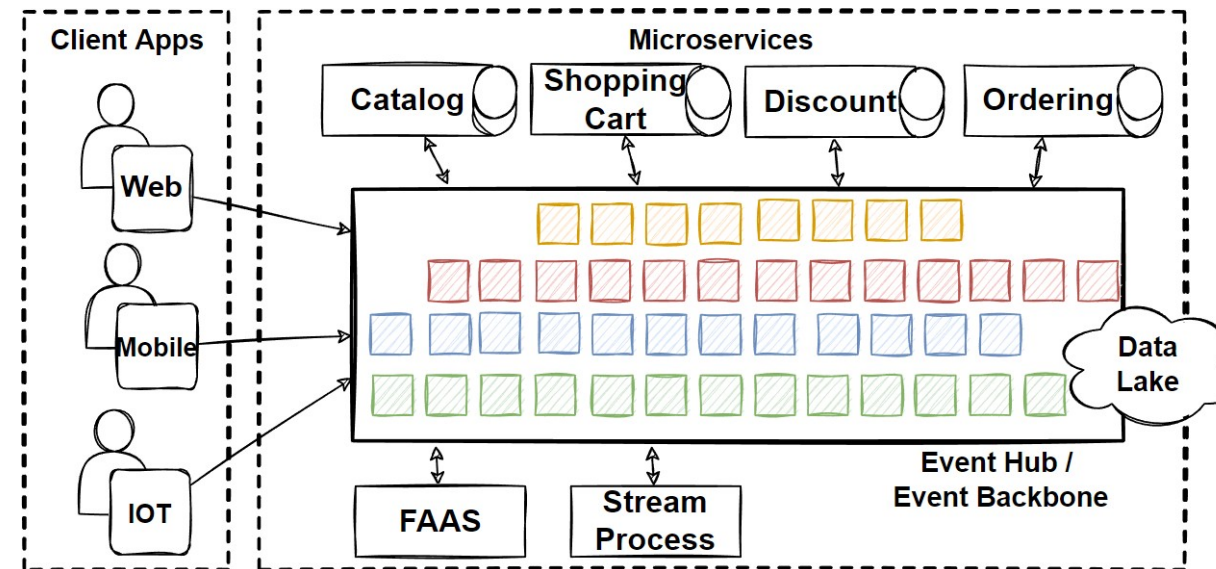
- Event-driven architecture comes with latency when publishing and subscribing events from the Event Hub.
- Sync REST APIs communication make expensive calls to a database that reduce performance.
- How can we make more faster that increase performance of communications in Microservices Architecture ?

Problems

- Slowness and Low Performance Communication
- Latency when publishing and subscribing events
- Rest APIs make Database calls that are expensive, low performance

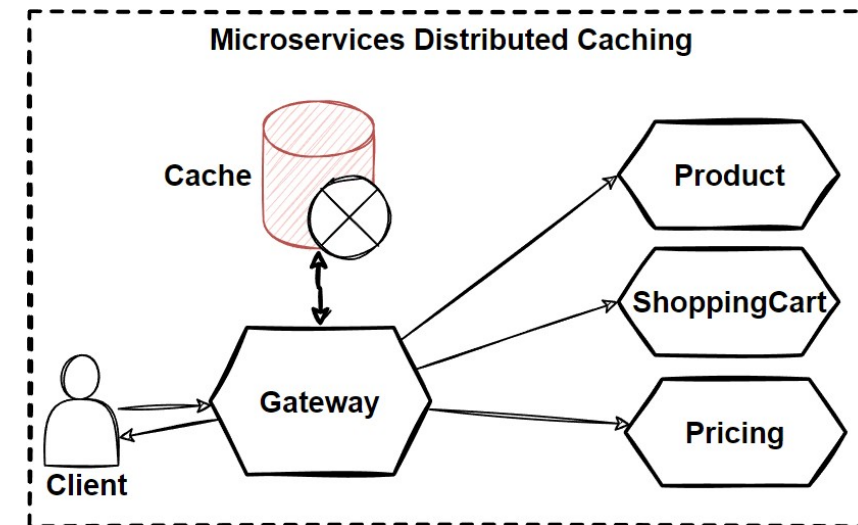
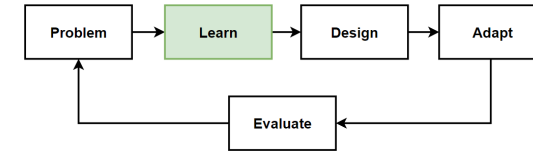
Solutions

- Distributed cache
- Storing frequently accessed data in a distributed cache



What is Caching ?

- **Caching** for improving the **performance** of a system **by storing frequently accessed data** in a **cache** that can be **quickly accessed** from **memory**.
- **Caching** is to **reduce** the **number of expensive operations**, such as **database queries** or **network requests**.
- **Caching** can **increase performance, scalability, and availability** for microservices with **reducing latency** with cache and makes application **faster**.
- When the **number of requests are increased**, caching provide to **handle requests** with **high availability**.
- If application **request** mostly comes for **reading data** that is not **changes so frequently**, then **Caching** will be so **efficient**.
- I.e. **reading product catalog** from e-commerce application. Caching also provide to **avoid re-calculation** processes.
- By **storing frequently accessed data** in a cache, system can **avoid** the **overhead of repeatedly expensive operations**.



Types of Caching

- **In-memory cache**

Stores data in the main memory of a computer. In-memory caches are typically the fastest type of cache, but the data is lost when the cache is restarted or the machine is shut down.

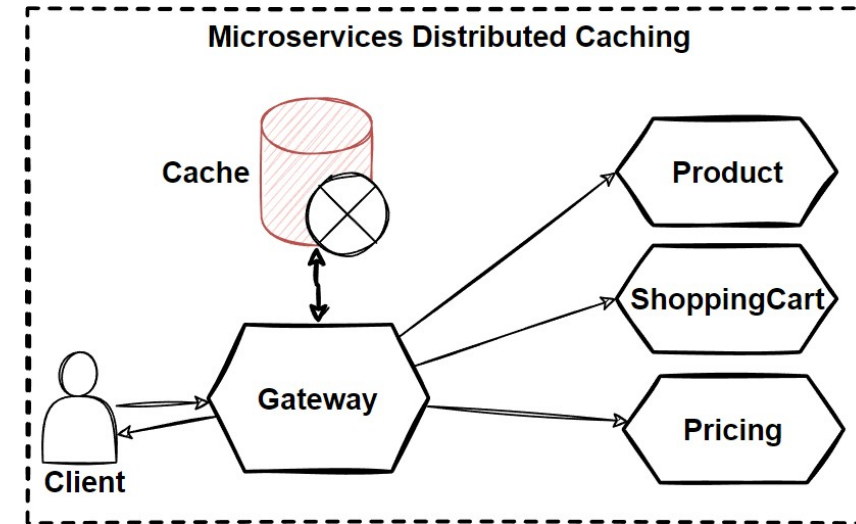
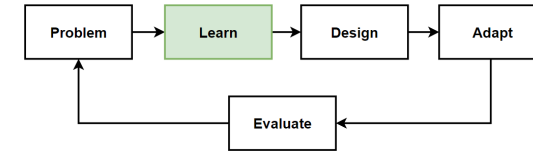
- **Disk cache**

Stores data on a hard drive or solid-state drive. Disk caches are slower than in-memory caches, but they can persist data.

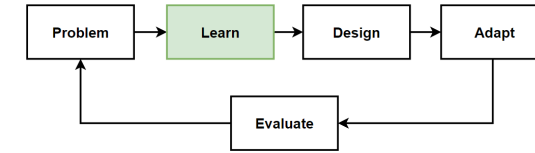
- **Distributed cache**

Cache is distributed across multiple machines and is typically used in distributed systems, such as microservices architectures.

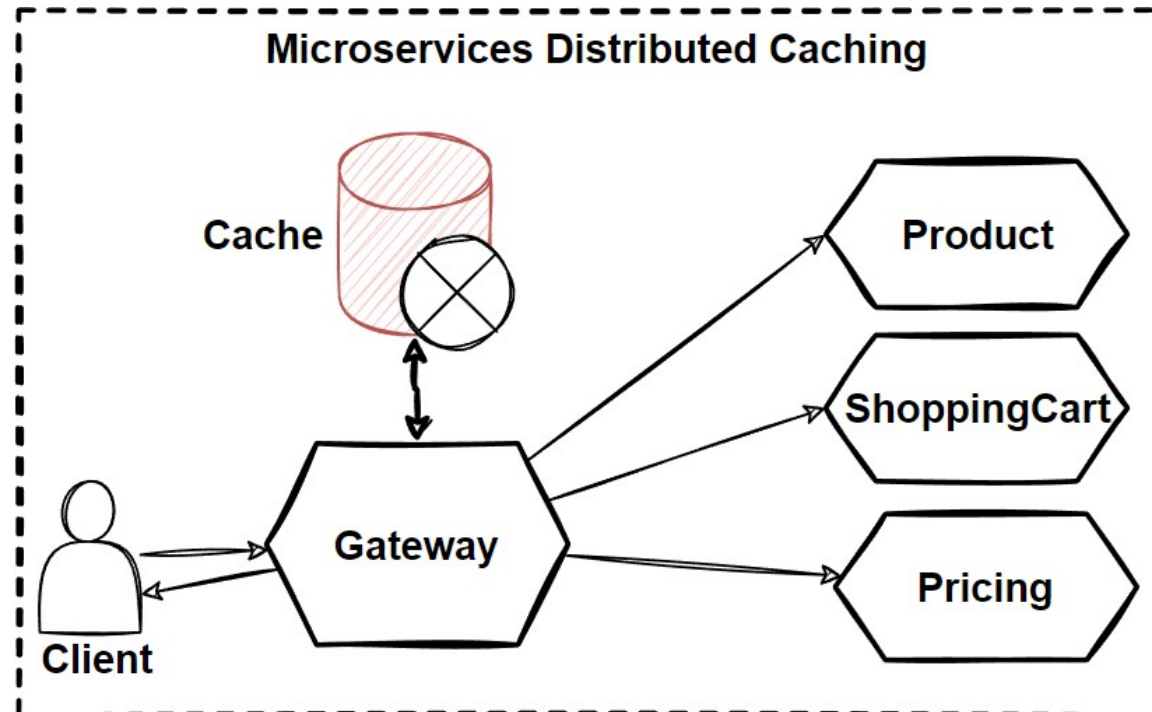
- Distributed caches can improve the performance and scalability of a system by allowing data to be stored and accessed from multiple locations.



Distributed Caching in Microservices

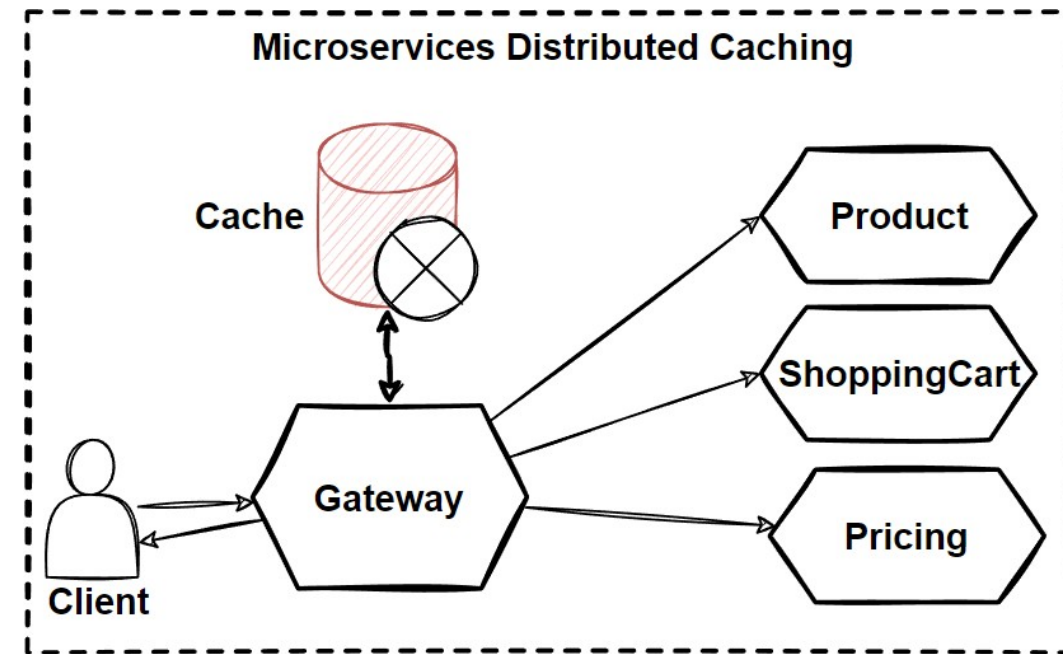
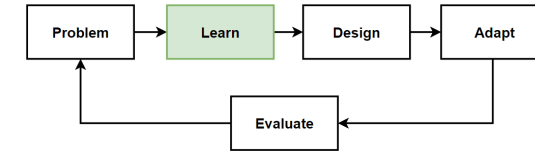


- **Distributed caching** is improving the performance by **storing frequently accessed data** in a **cache** that can be **quickly accessed** from **multiple locations**.
- **Microservices architectures** are typically implement a **distributed caching** architecture:
 - Improve the performance of individual services by storing frequently accessed data locally.
 - Reducing the need to make expensive calls to a database or other external system.
- **How can we increase the speed of the microservices ?** With using **Distributed Cache**.



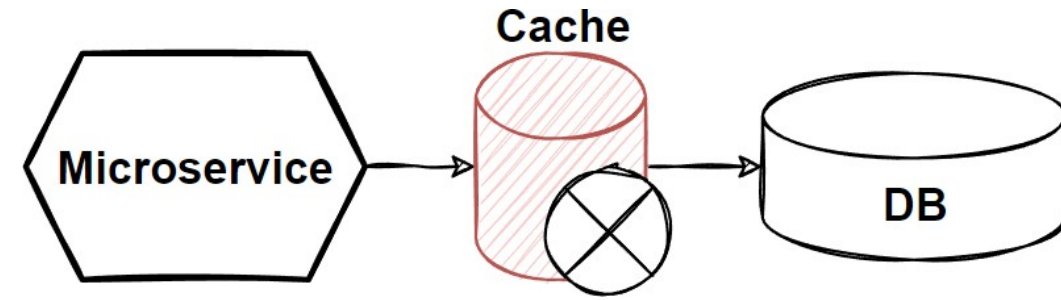
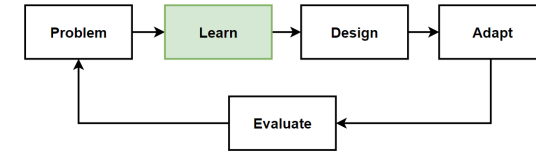
Distributed Caching in Microservices - 2

- Microservices are **responsible for a specific function** and **communicates** with other services through well-defined interfaces, **typically using APIs**.
- By **storing frequently accessed data locally in a cache**, microservices can **avoid the overhead of making repeated calls** to an external system, resulting in **faster response** times.
- Benefits to using distributed caching in a microservices:
 - Improved performance**
Services can avoid the overhead of making repeated calls to a database or other external system.
 - Resilience**
Allowing services to continue functioning even if an external system becomes unavailable.
 - Scalability**
Allowing services to handle increased traffic without the need to scale up the external system.

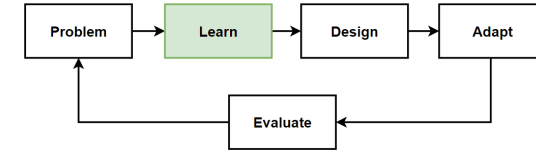


Cache Hit and Cache Miss

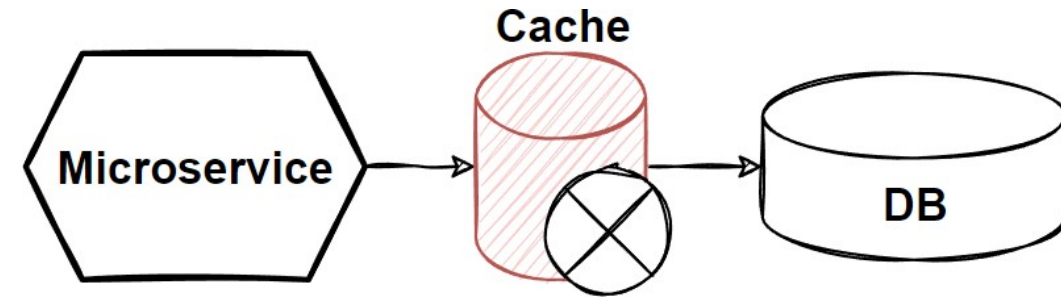
- **Cache hit** occurs when the **requested data** can be **found** in the **cache**.
- **Cache miss** occurs when the **requested data is not** in the **cache** and **must be retrieved** from a slower storage db.
- **Cache hits** are **desirable** because they can improve the performance of a system by **reducing** the **number of requests**.
- **Cache misses** can have a **negative impact** on **performance**, because they **require additional time** and resources to retrieve the requested data.
- **The cache hit rate** is a measure of **how often a cache** is able to **fulfill requests** from its own store.
- **High cache hit rate** indicates that the cache is **effective** at storing frequently accessed data.
- **Low cache hit rate** may indicate that the **cache is not large enough**.



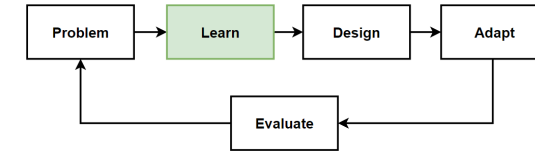
Caching Strategies in Distributed Caching



- There are **several caching strategies** that can be used in distributed microservices:
 - **Cache Aside**
 - **Read-Through**
 - **Write-Through**
 - **Write-Back, Write-Behind**
- **Cache Aside Strategy**
Client checking the cache for data before making a request to the backend service. When microservices needs to read data from the database, it checks the cache first to determine whether the data is available.
- If the **data is available (a cache hit)**, the cached data is returned. If the **data isn't available (a cache miss)**, the database is queried for the data.
- The client will **retrieve the data from the backend service** and store it in the cache for future requests.
- Data is **lazy loaded into cache** by client application.



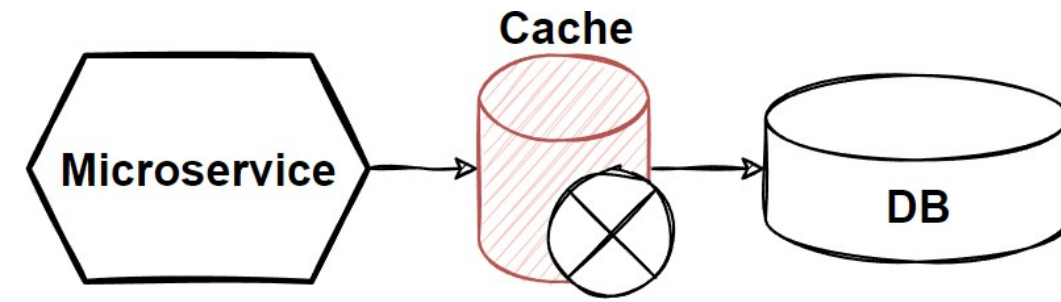
Caching Strategies in Distributed Caching



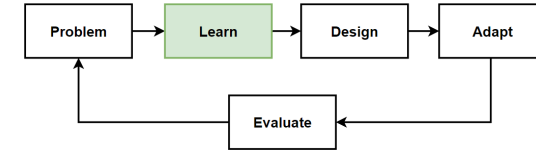
- **Read-Through Strategy**

When there is a cache miss, it loads missing data from the database, populates the cache and returns it to the application.

- When a client **requests data that is not found** in the cache, the cache will **automatically retrieve** the data from the underlying database and **store** it in the **cache** for future requests.
- **Cache-aside strategy**, when a client requests data that is **not found in the cache, the client is responsible** for retrieving the data from the database.
- **Read-through cache strategy**, when a client requests data that is **not found in the cache, the cache will automatically retrieve** the data from the database.
- **Cache** always **stays consistent** with the database.



Caching Strategies in Distributed Caching



- **Write-Through Strategy**

Update the cache whenever data is written to the backend service. Cache always has the most up-to-date data, but it can also result in a higher number of write operations.

- **Instead of lazy-loading** the data in the cache after a cache miss, the cache is proactively updated immediately following the primary database update.

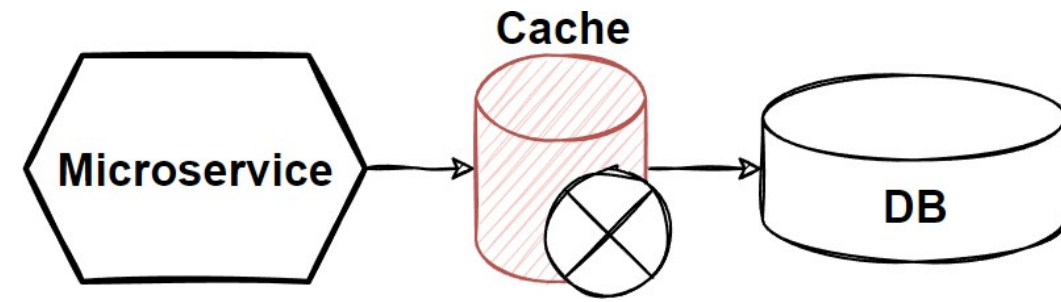
- Data is first written to the cache and then to the database.

- **Write-Back or Write-Behind Strategy**

Delays updating the cache until a later time. This reduce the number of write operations, but the cache may not have the most up-to-date data.

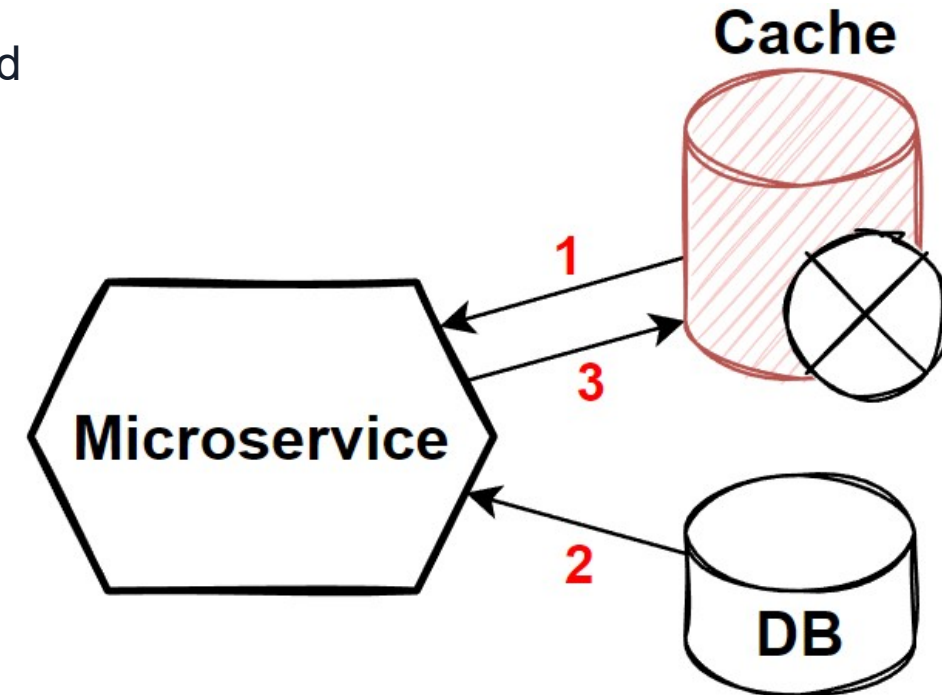
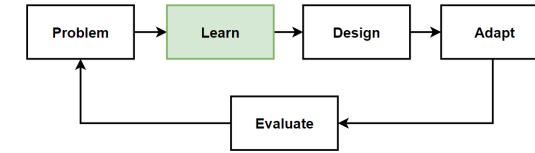
- **In Write-Through**, the data written to the cache is synchronously updated in the main database.

- **In Write-Back or Write-Behind**, the data written to the cache is asynchronously updated in the main database.

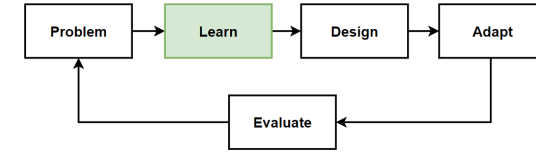


Cache-Aside Pattern for Microservices

- (1) When a client needs to access data, it first checks to see if the data is in the cache.
- (2) If the data is in the cache, the client retrieves it from the cache and returns it to the caller.
- (3) If the data is not in the cache, the client retrieves it from the database, stores it in the cache, and then returns it to the caller.
- Some of caching systems provide **read-through** and **write-through/write-behind** operations. In these systems, **client application retrieves data over by the cache**.
- For not supported Caches, it's the **responsibility the applications use the cache** and **update the cache** if there is a **cache-miss**.
- **Microservices** good example to implement **Cache-Aside pattern**, it is common to use a **distributed cache** that is **shared across multiple services**.



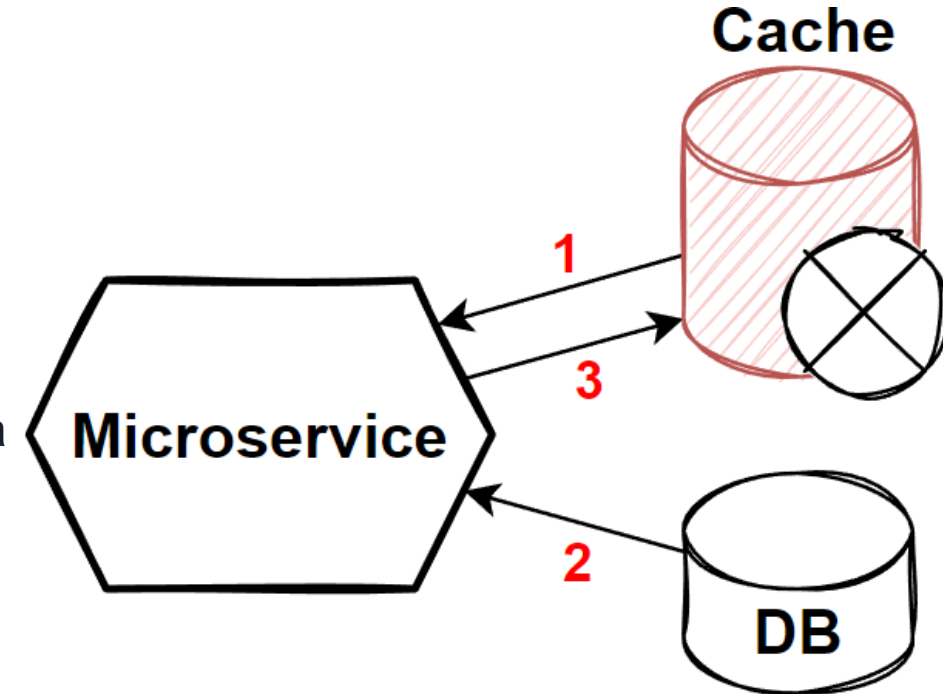
Cache-Aside Pattern for Microservices - 2



- **Cache-aside pattern** can **improve performance** of a **microservices architecture**, by **reducing the number of expensive database calls**.
- To use the Cache-aside pattern in a microservice, **need to implement a cache layer** in your service.
- Involve using a **Cache library** or framework, such as **Redis** or **Memcached**, or implementing a custom cache solution.

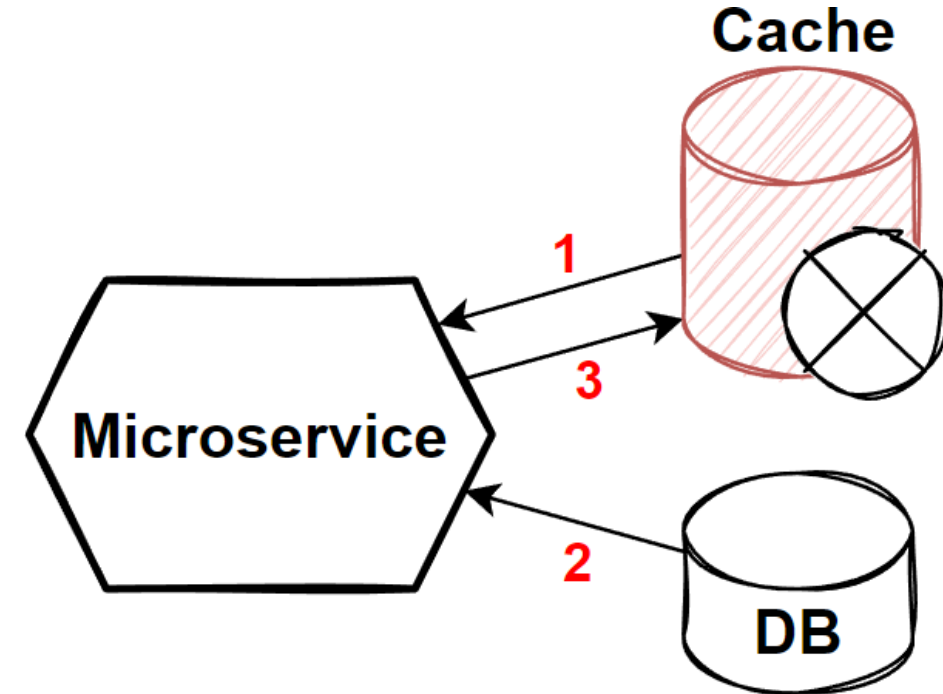
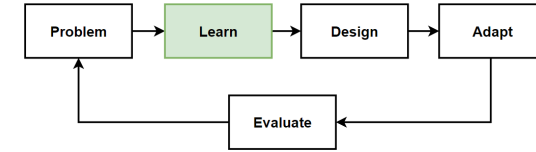
Process:

- When a service needs to access data, it first checks to see if the data is in the cache.
- If the data is in the cache, the service retrieves it from the cache and returns it to the caller.
- If the data is not in the cache, the service retrieves it from the database or other data store, stores it in the cache, and then returns it to the caller.



Drawbacks of Cache-Aside Pattern for Microservices

- **Cache** can introduce **additional complexity** and may not be suitable for all situations.
- The cache may **need to be invalidated** or **refreshed** when data is updated in the database or data store.
- This can **require additional coordination** between the microservices.
- The cache may introduce **additional latency** if it is located remotely from the microservices that are using it.



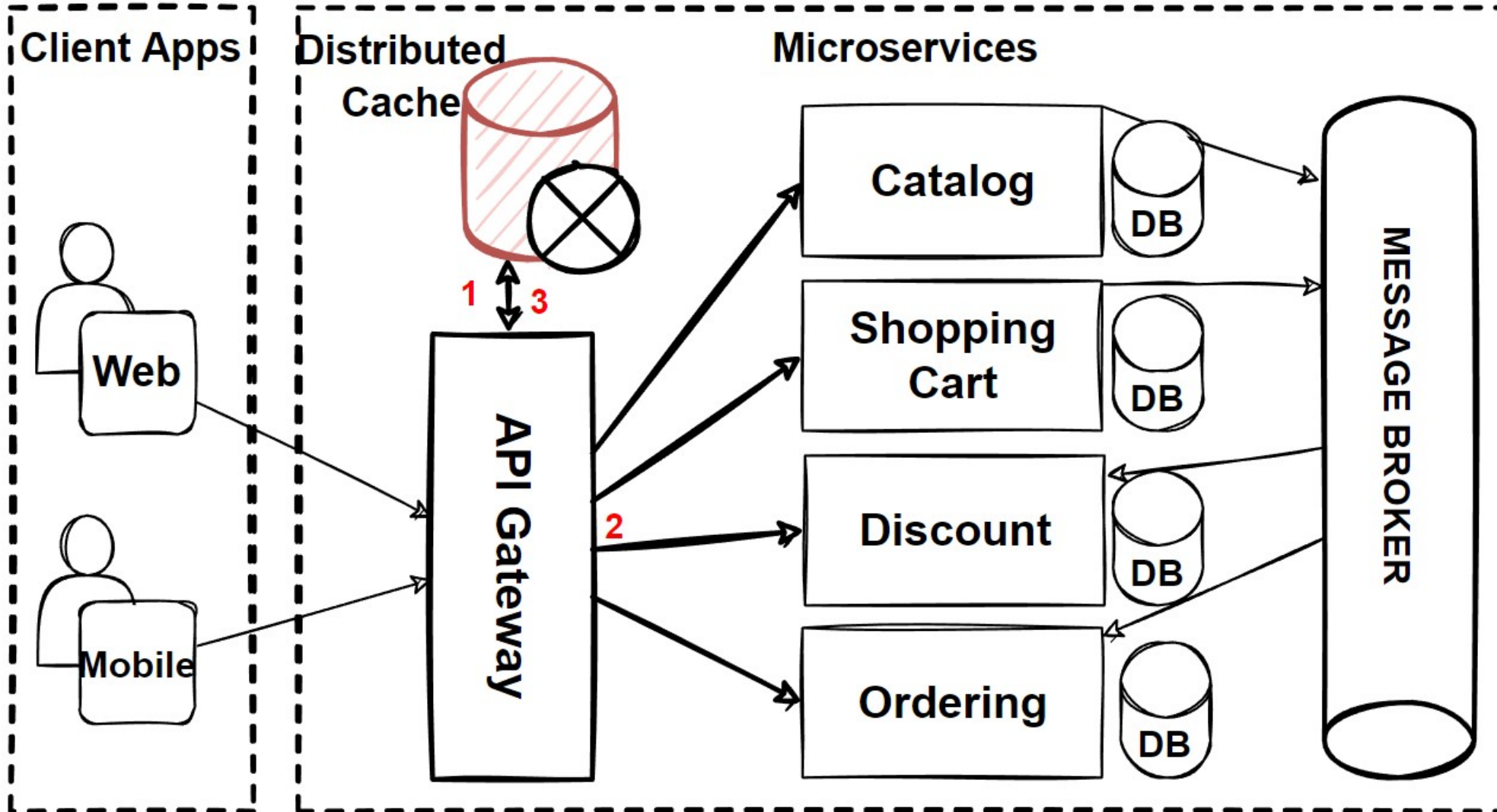
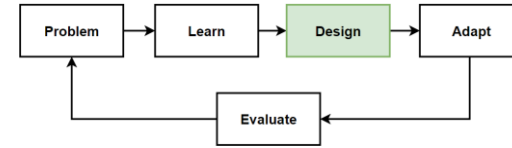
Before Design – What we have in our design toolbox ? - Old

Architectures	Patterns&Principles	Microservices EDA	Non-FR	FR
• Microservices Architecture • Event-Driven Microservices Architecture	• The Database-per-Service Pattern • Polygot Persistence • Decompose services by scalability • The Scale Cube • Microservices Decomposition Pattern • Microservices Communications Patterns • Microservices Data Management Patterns • Event-Driven Architecture	• Asynchronous, Decoupled communication • Event Hubs • Stream-Processing • Real-time processing • High volume events	• High Scalability • High Availability • Millions of Concurrent User • Independent Deployable • Technology agnostic • Data isolation • Resilience and Fault isolation	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history

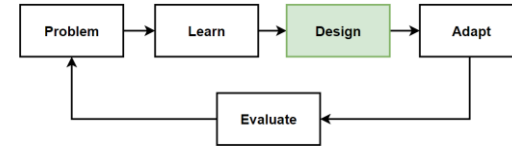
Before Design – What we have in our design toolbox ? - New

Architectures	Patterns&Principles	Microservices Caching	Non-FR	FR
• Microservices Architecture • Event-Driven Microservices Architecture	• The Database-per-Service Pattern • Polygot Persistence • Decompose services by scalability • The Scale Cube • Microservices Decomposition Pattern • Microservices Communications Patterns • Microservices Data Management Patterns • Event-Driven Architecture • Microservices Distributed Caching	• Caching Strategies • Cache Invalidation • Cache Hit - Cache Miss • Cache-Aside Pattern	• High Scalability • High Availability • Millions of Concurrent User • Independent Deployable • Technology agnostic • Data isolation • Resilience and Fault isolation	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history

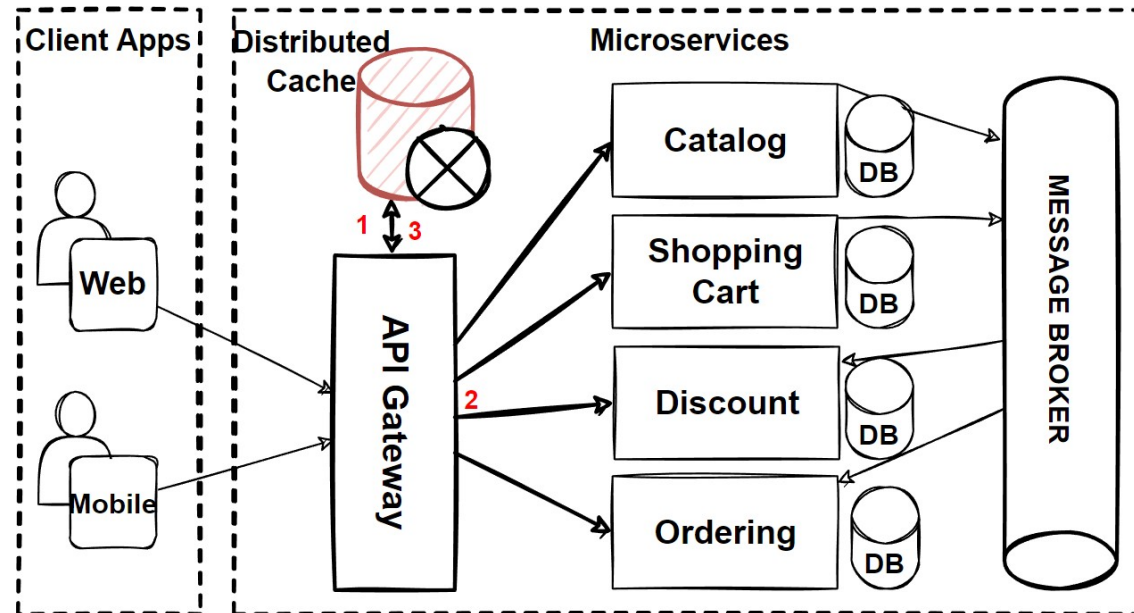
Microservices Distributed Caching with Cache-Aside Pattern



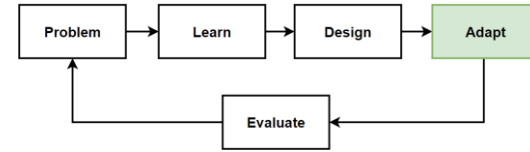
Microservices Distributed Caching apply Cache-Aside Pattern on API Gateway



1. User makes a request to an API gateway to retrieve some data.
2. API gateway determines which microservice is responsible for handling the request and receives the request and checks to see if the data is in the cache.
3. If the data is in the cache, API gateway retrieves it from the cache and returns it to the client.
4. If the data is not in the cache, API gateway forward responsible microservice to retrieves it from the database, and then returns it to the API gateway.
5. The API gateway receives the data from the microservice and stores it in the cache and returns it to the user.



Adapt: Microservices Distributed Caching



Frontend SPAs

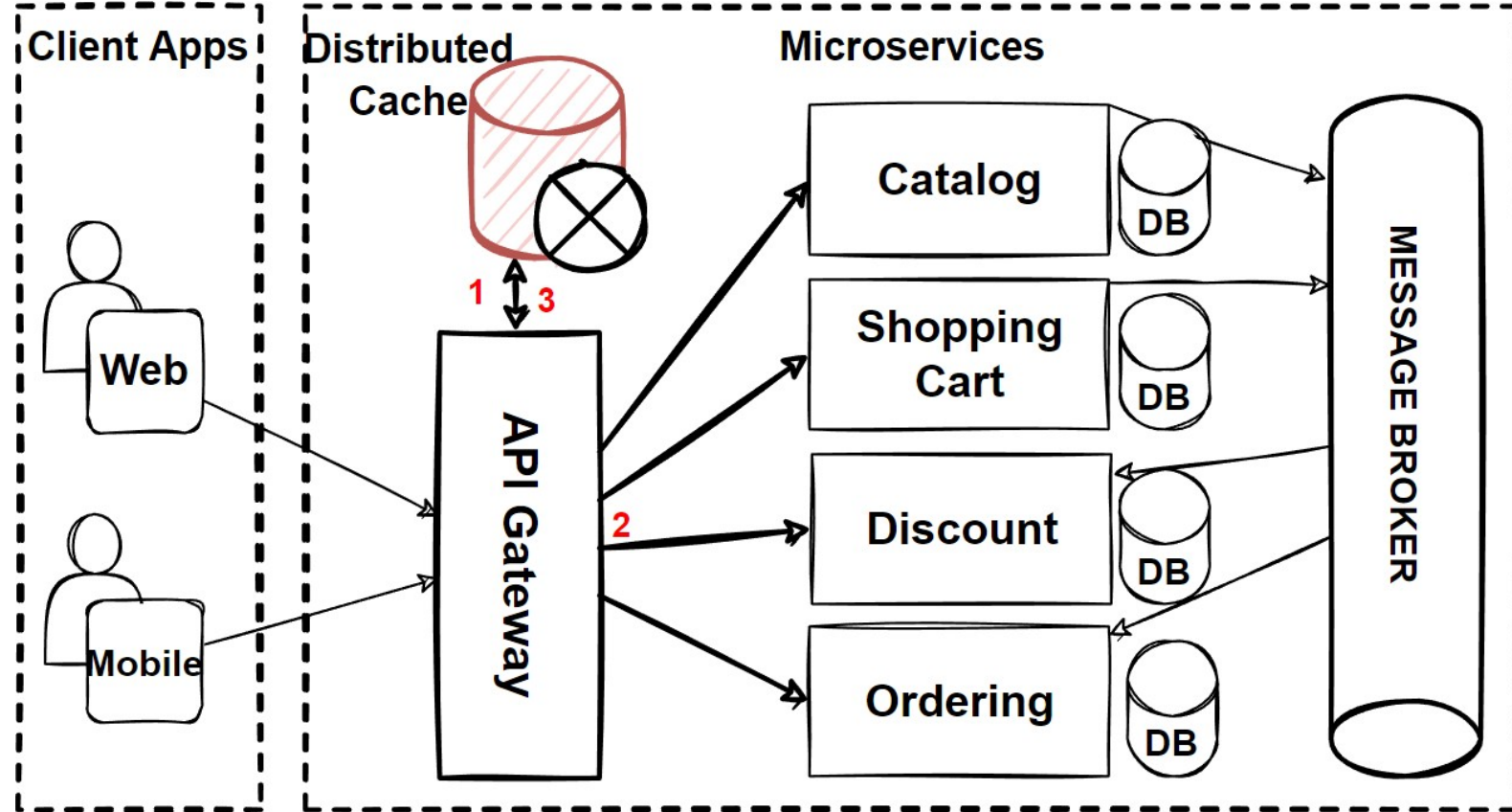
- Angular
- Vue
- React

API Gateways

- Kong Gateway
- Express Gateway
- Amazon AWS API Gateway

Backend Microservices

- Java – Spring Boot
- .Net – Asp.net
- JS – NodeJS



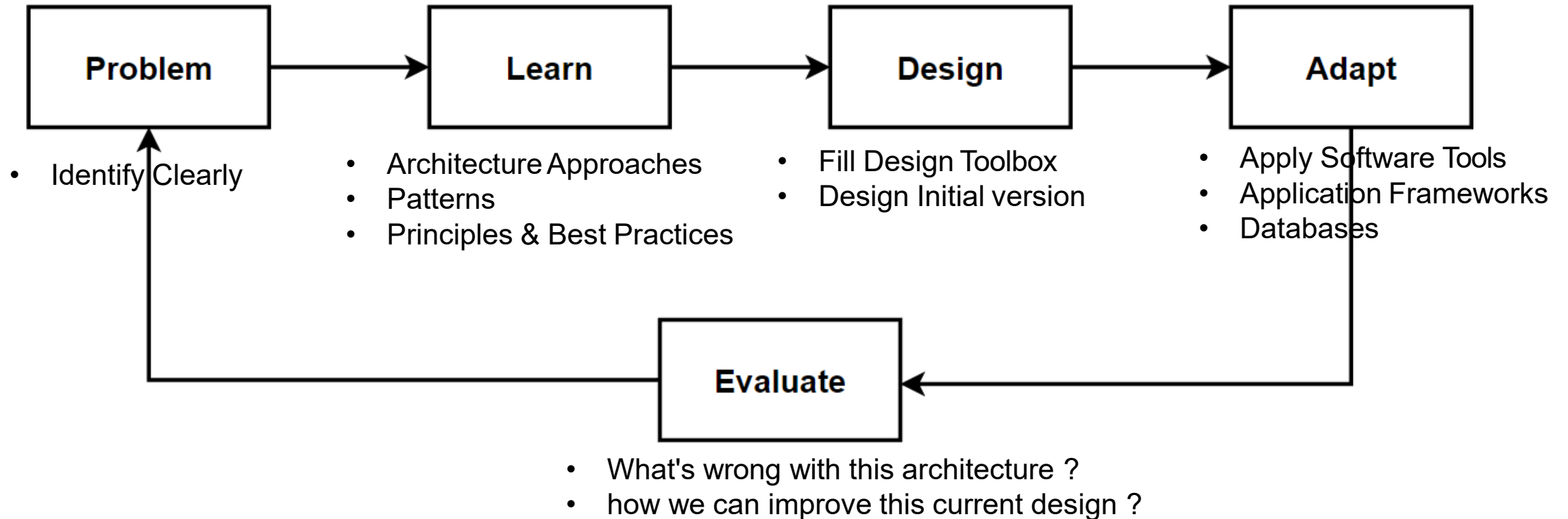
Open Source Distributed Caches

- Redis
- Memcached

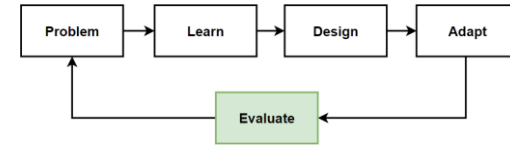
Cloud Distributed Caches

- Amazon ElastiCache
- Azure Cache for Redis

Way of Learning – The Course Flow



Evaluate: Microservices Distributed Caching

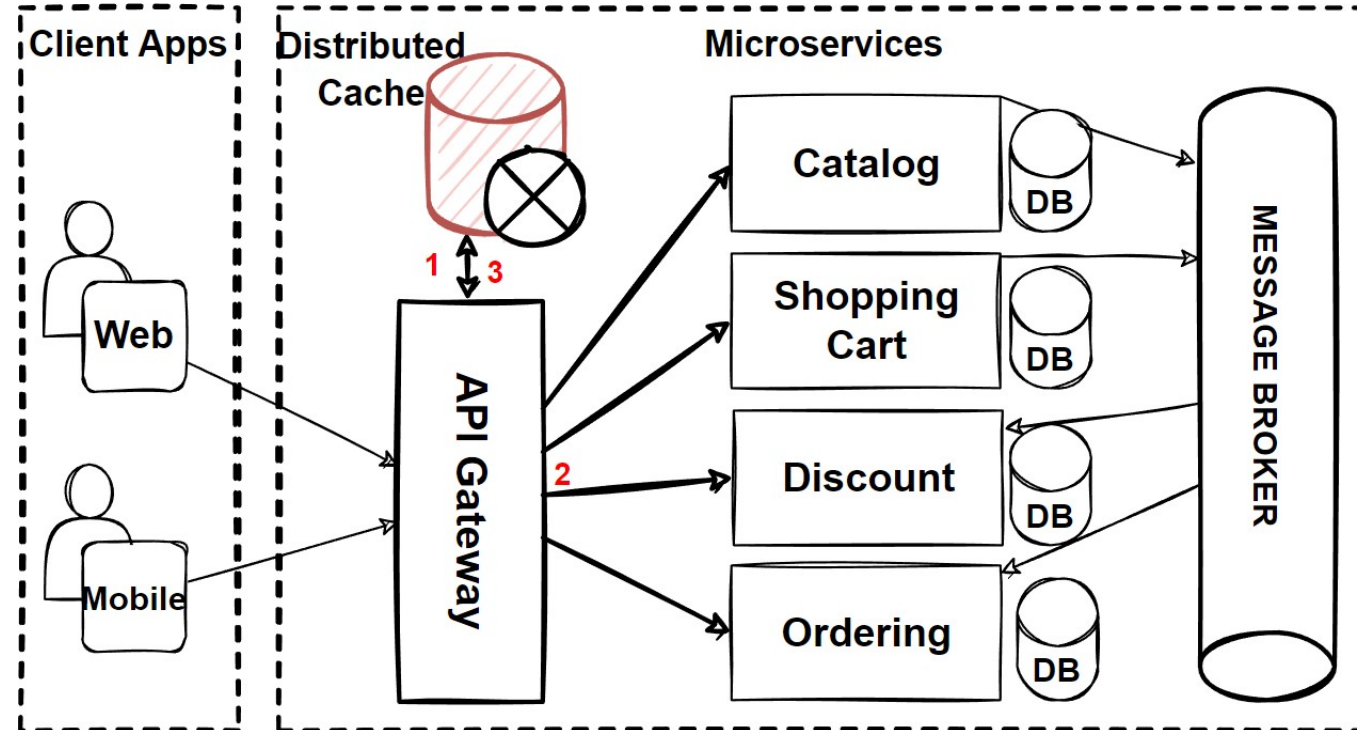


Benefits

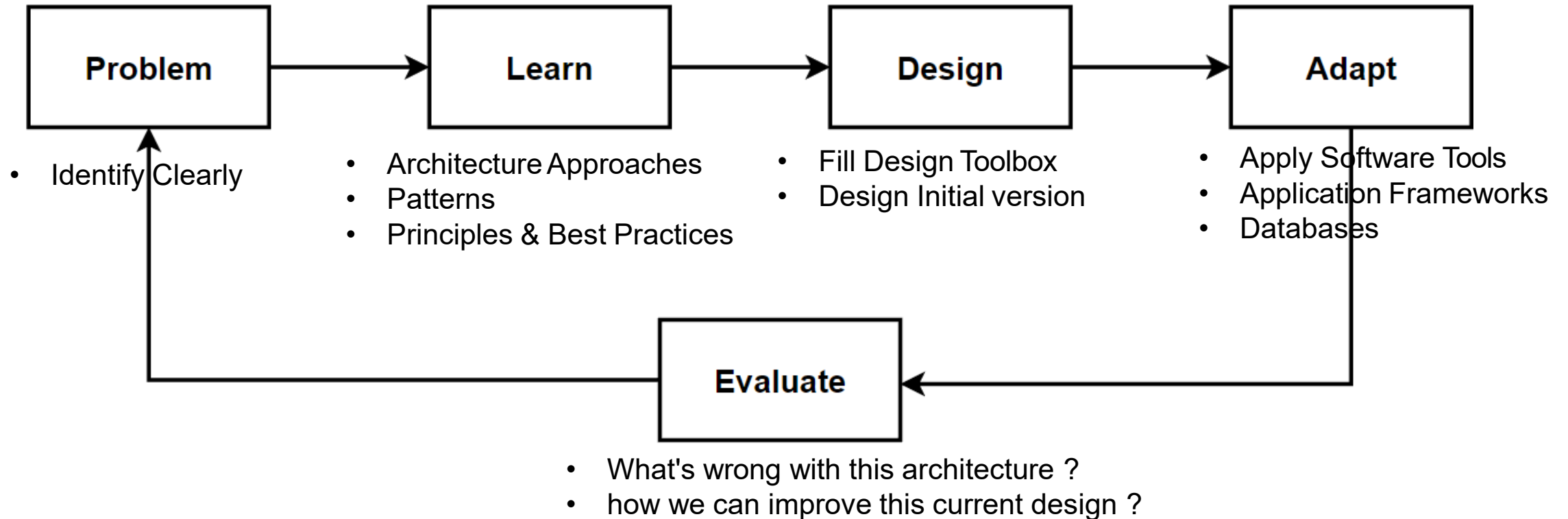
- **Improved performance;** by storing frequently accessed data in a cache
- **Enhanced scalability;** by offloading some of the workload from the database
- **Increased reliability;** cache can continue to serve data even if one of the nodes fails.

Drawbacks

- **Increased Complexity;** cache can be complex and require specialized knowledge.
- **Cache invalidation;** Maintaining the consistency of the cache can be challenging, can lead to issues.
- **Cost;** managed services from cloud providers, can be expensive
- **Cache stampede**



Way of Learning – The Course Flow



Problem: Deploy Microservices at Anytime with Zero-downtime and flexible scale

Problems

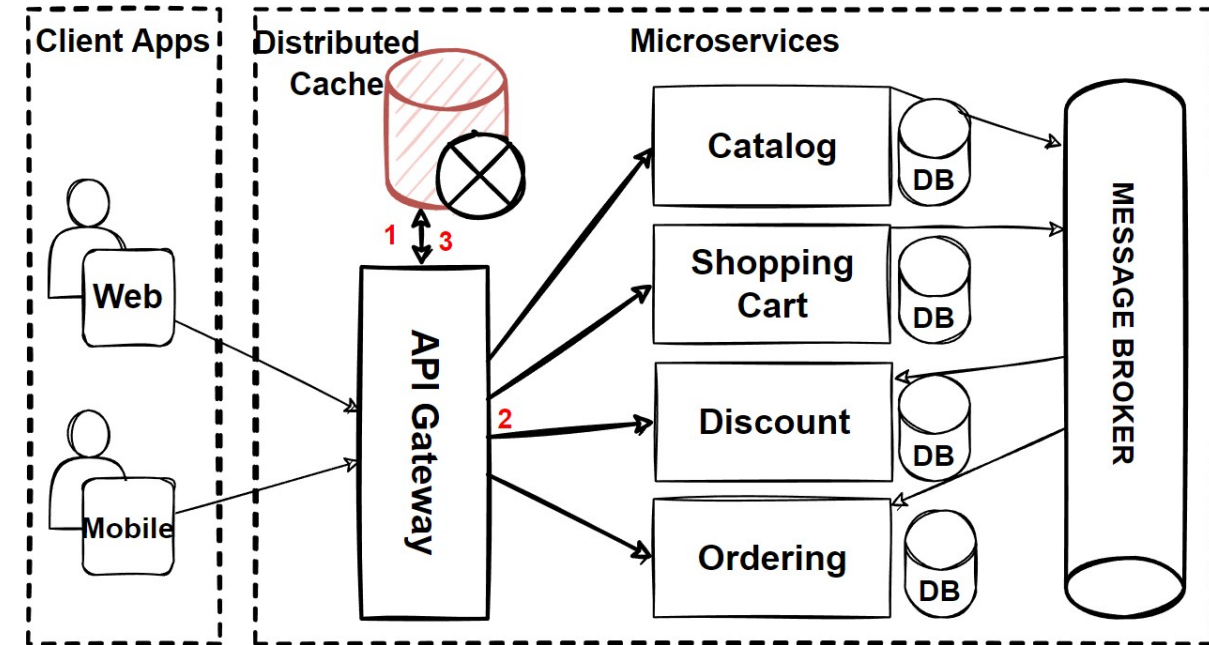
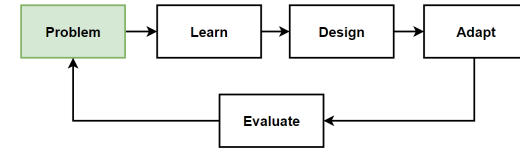
- Business teams wants to add new features immediately
- Innovate and experiment with new features
- Deploy features immediately, not waiting for deployment dates.
- Flexible scale for market peak times

Considerations

- Ensure continuity of service and minimize disruption
- Allow for continuous delivery
- Support high-traffic environments

Solutions

- Containers and Orchestrators
- Deployment strategies; blue-green deployment, rolling deployment, and canary deployment.
- Kubernetes Patterns; Sidecar Patterns, Service Mesh Pattern
- DevOps and CI/CD Pipelines and Infrastructure as code (IaC)



**CÁM ƠN ĐÃ CHÚ Ý
LẮNG NGHE!**